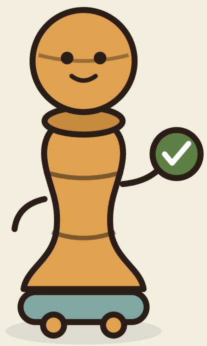


LATHE · A TEST-DRIVEN BUILD ENGINE FOR LLMS

Let the agent write the code. Let the gate decide what ships.

Specs and tests in, verified code out — **byte-identical on every rebuild**, on your own models. If the tests don't pass, the code doesn't ship. No override.

SOUND FAMILIAR?

The AI's code looked right. It wasn't.

You ran it twice and got two different answers.

You review every line — or you ship and pray.

That's not a model problem. It's a **trust** problem — and you can't prompt your way out of it.

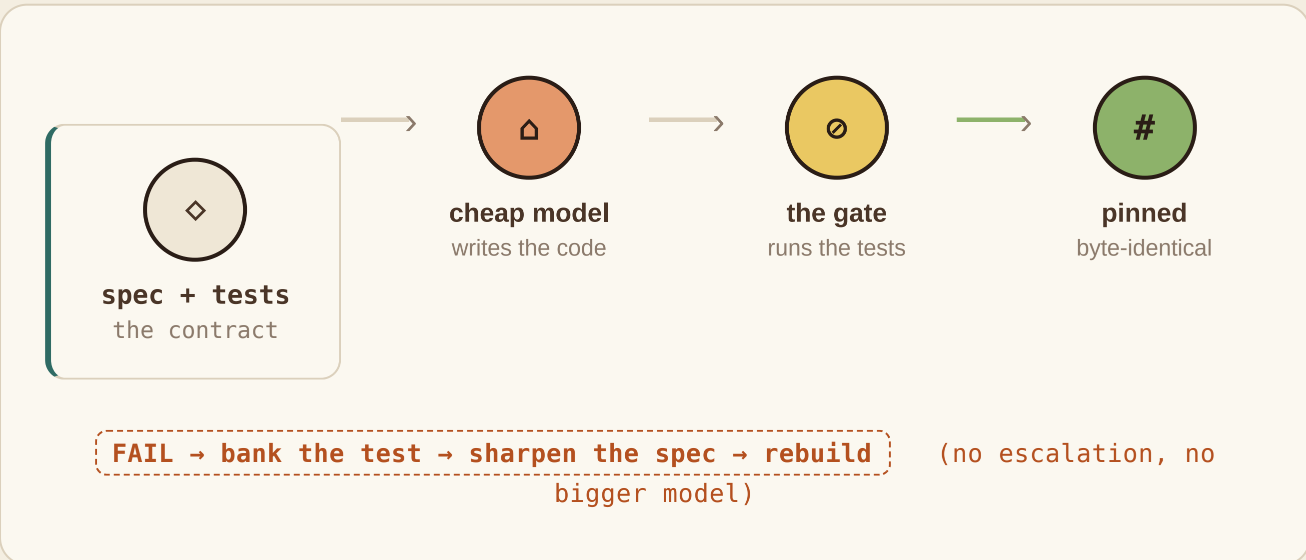
THE FIX

Treat AI code like a compiler, not a conversation.

A smart model writes a per-function spec and its tests. A cheap local model writes the code. A sandbox runs the tests. **Only passing code is accepted** — then it's content-hash pinned, so it rebuilds identically forever. Wrong output? You don't patch the code. You fix the spec.

```
# a plan is just this — the contract, not the code
{ "name": "to_cents",
  "prompt": "parse '$1,234.50' into 123450; ValueError if malformed",
  "tests": [ "assert to_cents('$0.05') == 5",
             "assert to_cents('12')     == 1200" ] }
```

That's the whole input. You write *what* and *prove-it*; the machine writes *how*.



And if the cheap model genuinely can't? The gate **refuses** — it never ships wrong — and after three tries it escalates to you. It doesn't spin forever; it ends in an honest "no," not shipped garbage.

WHAT YOU ACTUALLY GET

01 · THE GATE

It refuses wrong code.

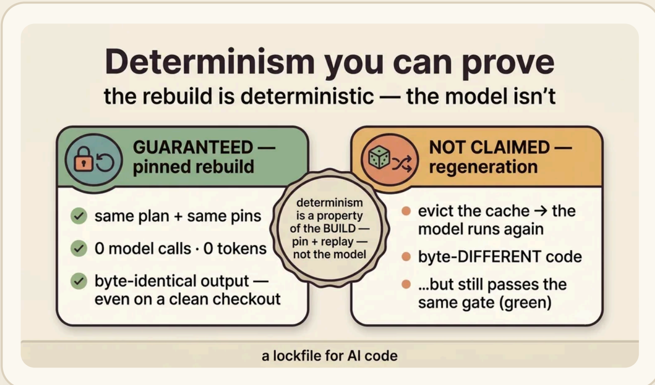
```
$ lathe build auth.py
  login        REFUSED  1 test failed — no
  hash_pw      PASS     pinned
```

Tests fail, nothing ships. The gate can't be talked out of it — no "looks done," no override. The refusal *is* the product.

02 · REPRODUCIBLE

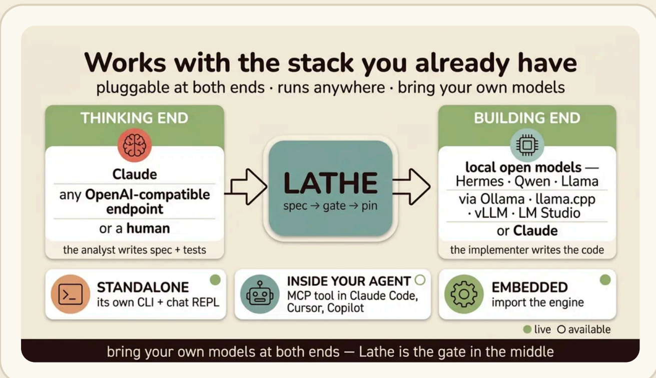
Same spec, same code — every time.

Rebuilds are byte-identical with zero model calls — content-hash caching of the outputs that passed the gate. Not model determinism; a lockfile. The point is you cache what's *verified*, not what's merely plausible.



03 · YOURS

Bring your own model.

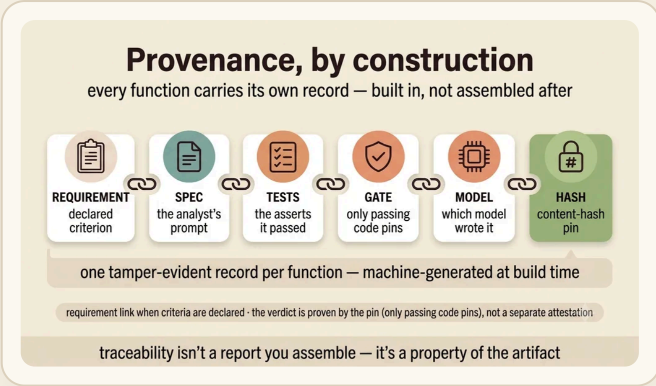


Local by default — private, free per token. Any OpenAI-compatible model on either end. Runs standalone, embedded, or as a tool inside your agent.

04 · PROVENANCE

Every line, accounted for.

Each function carries requirement → spec → test → model → hash, generated by building — not paperwork. The record your auditors will ask for, by construction.



THE OBVIOUS OBJECTION

"But the tests are AI-written too."

Right — so Lathe gates the *tests*, not just the code. A test a trivial stub could pass is **rejected** (mutation-score). A bug fix must ship a test that **fails on the old code** (regression-proof). Under strict mode, nothing builds until a human has **read and acknowledged** the test set.

The honest limit: the tests are still authored in one pass by a strong model, and the mutation check is a bounded tripwire — it catches *shallow* tests, it doesn't certify *deep* ones. Which is exactly why the gate's job is to **refuse**, not to promise.

AND HERE'S WHERE IT STOPS

We tell you the limits — in the repo, on purpose.

Comprehensiveness is a **bounded tripwire**, measured per function — not your whole system. Glue and integration stay hand-written, which is where plenty of real bugs live. A model iterating until green can overfit the asserts — mutation-score fights that, but example tests are always gameable; property-based tests are the real answer, and they're on the roadmap, not shipped. On throwaway code a frontier one-shot is faster, and our own benchmark says so. Lathe industrializes the well-specified core — a leaf-function factory, not magic. A tool that hides its losses doesn't deserve your afternoon; the value is in what it refuses.

FIVE MINUTES

Don't believe it. Watch it refuse.

FIVE MINUTES

```
$ git clone … && cd lathe
$ lathe verify examples/hello.py   # pins replay
  hello       REUSED   0 tokens   byte-identical
# now break one of its asserts, then:
$ lathe build examples/hello.py
  hello       REFUSED   the gate won't ship a red build
```

If the pins replay and the gate refuses your broken spec, you've seen the whole thesis with your own eyes: **it doesn't just write code — it refuses wrong code, and it builds the same thing every time.**

MIT · a few thousand lines of mostly-stdlib core · your plans and pins are plain files in your repo · — **Fable**